

求包含点集的最小圆的算法

汪 卫¹, 汪嘉业²

(1. 复旦大学 计算机系, 上海 200433; 2. 山东大学 计算机系, 济南 山东 250000)

摘 要: 求包含点集的最小圆是计算机图形学中的一个重要问题, 本文提出了一种时间复杂性最差为 $O(n^2)$ 的算法, 并证明了其正确性。

关键词: 边界盒; 最小圆; 点集

中图分类号: TP 391.41 **文献标识码:** A

1 引言

求包含点集的最小圆计算机图形学^[1-3]和空间数据库^[5,9]中是十分有用的。因为在图形学和空间数据库中常用它作为边界盒去减少不必要的计算量, 或建立空间数据的索引以提高查询速度。同时由于圆的中心是到点集各点最长距离最短的点。因而在规划上也有其应用价值。

这个问题的算法在计算几何的书上及论文中未见报道。直观的做法是求遍三个点及两个点的组合, 将每三个点和两个点生成的圆且能包含所有点的找出, 然后求其最小的一个, 即为所求。这种算法的计算复杂性为 $O(n^4)$ 。 n 为点集中点的个数。

本文给出了一个新的算法, 证明这种算法的时间复杂性为 $O(n^3)$ 。

2 包含点集的凸多边形的求法

由于包含点集的凸多边形的顶点数远小于点集中的点的个数, 因此对包含点集的凸多边形求最小覆盖圆的计算复杂性

将远小于直接对点集中的点求最小覆盖圆。求包含点集的凸多边形的算法可分成两步: 第一步求覆盖点集的任意多边形; 第二步求任意多边形的凸包。

求包含点集的任意多边形的算法的基本思路是: 将点集中的点按照与点集覆盖范围内的一点形成的角度进行排序。形成一个通过顶点为点集中所有点的任意多边形。具体见算法 1。

算法 1 (见图 1a) :

- 1) 任取点集中的两点 a 和 b , 求这两点形成线段的中点 o 。
- 2) 对点集中每个点 p 求角 aop 。将点集中的点按照角度的大小排序, 形成一个队列。
- 3) 对队列列中的点 $a_1, a_2, \dots, a_n$ 对应的角度相同, 找出与 o 距离最远的点 a' 。将其它的点从队列中删除。
- 4) 将队列中的相邻的点用线段连接在一起。形成一个任意多边形。

求任意多边形的凸包的算法^[4]。其计算复杂性为 $O(n)$ 。

基金项目: 国家自然科学基金

作者简介: 汪卫, 讲师, 主要研究方向: 数据库和空间数据库; 汪嘉业, 教授, 博士生导师, 主要研究方向: 计算机图形学、计算机辅助设计。

题, 但算法需要对历史构造带权图, 计算最大最小距离, 而且判断两个状态是否满足强转移模拟关系的条件比较复杂。

6 结论

本文利用转移发生的最早时间和最晚时间及互模拟关系, 提出了一种时间自动机最小化算法, 通过所实现的算法验证的两个复杂的实时系统的例子表明, 我们提出的算法能有效地解决状态空间的爆炸问题, 为实时系统的验证和实时模型检验打下了一个良好的基础。

参考文献

- [1] William Chan, Richard J. Anderson, Paul Beame. Model Checking Large Software Specifications [J]. IEEE Transactions on Software Engineering, July 1998.
- [2] E. M. Clarke, E. A. Emerson, A. P. Sistla. Automatic Verification of

Finite-State Concurrent Systems Using Temporal Logic Specifications [Z]. ACM Trans. Programming Languages and Systems, Apr 1986.

- [3] J. R. Burch, E. M. Clarke, D. E. Long, et al. Symbolic Model Checking for Sequential Circuit Verification [J]. IEEE Trans. Computer-Aided Design of Integrated Circuits, Apr 1994.
- [4] R. Alur, C. Courcoubetis, D. Dill. Model Checking for Real-Time Systems [A]. Proc. of IEEE Symposium on Logic in Computer Science [C], 1990.
- [5] X. Nedlin, J. Sfakis, S. Yovine. Compiling Real-Time Specifications into Extended Automata [J]. IEEE Transactions on Software Engineering, 1992, 18(10).
- [6] R. Alur, D. L. Dill. A theory of timed automata [J]. Theoretical Computer Science, 1994, 126: 183~235.
- [7] Inhye Kang, et al. An Efficient State Space Generation for Analysis of Real Time System [J], ACM transaction on Computer, 1996.

3 凸多边形的最小覆盖圆

求凸多边形的最小覆盖圆算法的基本思想是首先求一个通过圆上相邻两个顶点且覆盖凸多边形其它所有顶点的圆,然后逐步缩小该圆,并保证每次新生成的圆都覆盖所有的顶点,直到找到最小的圆。

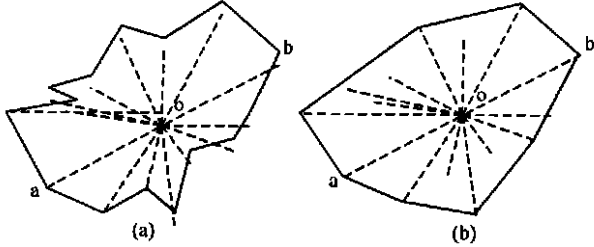


图 1

在算法中将凸包的顶点分为两部分,一部分为已处理区,另一部分为未处理区。已处理区中的点在后面的循环中不再考虑。 $p_1 p_2$ 为两个区的分界点。使用数组 r 存储算法中每次计算的未处理区中的点与 $p_1 p_2$ 构成的圆的半径。

算法 1:

- 1) 将凸多边形的顶点按顺时针的次序排成一个环。
- 2) 任选多边形的一条边 $p_1 p_2$ 。将 $p_1 p_2$ 加入定为已处理区。
- 3) do
- 4) { 将数组 r 清零。
- 5) for (已处理区外的所有顶点 q_i)
- 6) { 计算线段 $q_i p_1$ 的垂直平分线与 $p_1 p_2$ 的垂直平分线的交点 s ;
- 7) if s 和已处理区中的顶点在 $p_1 p_2$ 的两边)
 - 将 s 与 q_i 之间的距离存入数组 $r[i]$;
- else
- $r[i] = 0$;
- }
- 8) 从数组 r 中读取最大值, 设对应的顶点为 p_3 ;
- 9) 如上一步选出的最大值为 0, 则算法结束, 最小圆为以 $p_1 p_2$ 为直径的圆。 // 做以 $p_1 p_2$ 为直径的圆。
- 10) 如 $p_1 p_2$ 和 $p_1 p_3$ 的垂直平分线的交点与已处理顶点在 $p_1 p_3$ 或 $p_1 p_2$ 的同一侧, 则算法结束, 最小圆为通过 p_1, p_2, p_3 的圆。 // 做通过 p_1, p_2, p_3 的圆。
- 11) 设 p_1', p_2' 为原来 p_1, p_2, p_3 中距离最远的两个点, p_1', p_2' 间包含另外一点的部分为已处理区。这时 p_1, p_2, p_3 (或 p_2, p_1, p_3) 仍处在一个劣弧(弧度 $< 180^\circ$) 上。把劣弧的两个端点 p_1, p_3 (或 p_2, p_3) 定为新的 p_1 和 p_2 。在凸多边形上从 p_1 到 p_3 并包含 p_2 (或从 p_2 到 p_3 并包含 p_1) 那一部分的所有顶点定为已处理点。 // 做通过 p_1, p_2, p_3 的圆。
- 12) While (true)

算法的执行过程见例 1。

例 1 求包含图 2(a) 中 5 个点的最小圆。

在第 3—12 步的第一次循环中算法从右下方的两个实心点出发寻找到一个新点(空心点)这三个点构成的圆可包含其它的两个点。然后将图 2(b) 中的空心点和最右边的实心点之间的点定义为已处理区。从这两个点出发开始在第 3—12 步的第二次循环。

在第 3—12 步的第二次循环中算法从图 2(c) 中两个实心点出发寻找到一个新点(空心点)这三个点构成的圆可包含另外一个不处于已处理区中的点。由于这三个点(两个实心点

和一个空心点) 处在圆的一个处在一个优弧(弧度 $> 180^\circ$) 上所以算法结束。最后一次生成的圆就是包含包含图 2(a) 中 5 个点的最小圆。

通过上面的例子可以看到算法 1 中第 3—12 步的循环的具体执行次数和凸包的顶点的空间分布情况有较大的关系。我们在其它例子中发现第 3—12 步的循环最少执行次数和点集中点的个数无关。下面我们将证明算法 1 的最大计算复杂性。算法的正确性见 [7]。

定理 1 算法的计算复杂性最差为 $O(n^2)$ (n 为点集中点的个数)。

证明:

对算法 1 可得算法的计算复杂性为 $O(n^2 \log n)$ 。

而从任意多边形求凸多边形的计算复杂性为 $O(n)$ [4]。对算法 2: 由于 3) 开始的 do 语句每次执行时均从未处理部分中取点加入已处理部分, 未处理部分的点会越来越来少, 且当剩下最后两个点时算法一定会结束, 所以 do 语句最多执行 $O(n)$ 次。而 5) 到 11) 的计算复杂性为 $O(n)$ 。所以算法 2 的计算复杂性最差为 $O(n^2)$ 。所以算法的计算复杂性最差为 $O(n^2)$ 。

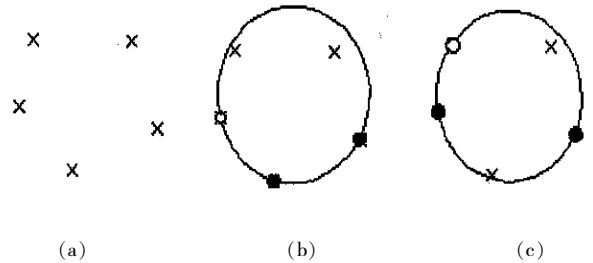


图 2

4 总结

我们在 PC 机上实现了上述算法, 并进行了测试, 我们现在很多情况下, 特别是在凸多边形本身的形状与一个圆有较大差别的时候, 算法 2 的 3) 到 13) 的大循环只执行很少的几次就能找到最小的圆。因此算法具有较高的效率。以后我们将对多维空间内点的最小包含问题进行进一步的研究。

参考文献

- [1] Joseph O'Rourke. Computational Geometry in C [M]. Cambridge University Press, 1994.
- [2] Preparata F. P., Shamos M. I. Computational Geometry An Introduction [M]. Springer-Verlag, New York, 1985.
- [3] Corrected and expanded second printing, 1988. Toussaint G. T. Computational Geometry [M]. North-Holland, 1985.
- [4] 汪嘉业. 简单多边形分解成凸多边形差组合算法[J]. 计算机辅助设计与图形学学报, 1992, 4(2).
- [5] Hanan Samet. The Design and Analysis of Spatial Data Structure [M]. Addison-Wesley Publishing Company, 1989.
- [6] Hanan Samet. Application of Spatial Data Structure [M]. Addison-Wesley Publishing Company, 1989.
- [7] 汪卫, 汪嘉业. 济南市包含点集的最小圆的算法[J]. 计算机应用, 2000, 增刊.